

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

SEDE REGIONAL DE RETALHULEU

FACULTAD DE INGENIERÍA EN SISTEMAS

CURSO: PROGRAMACIÓN III

DOCENTE: ING. YOEL MONZÓN ESTRADA.

TEMA

PROYECTO DE VISUALIZADOR INTERACTIVO DE ESTRUCTURAS DE DATOS Y ALGORITMOS.



Integrantes del Grupo de Trabajo 3.

Bryan Ricardo Rosales Morales

Gabriel Estuardo Polero Benavente

Hiker Ramiro Salazar Gramajo

Luis Eduardo Flores García

Roberto Arenas.

Carné:

2890-24-18409

2890-24-8909

2890-24-5757

2890-21-12987

2890-24-.

Ciudad de Retalhuleu, 30 de mayo de 2026.

DOCUMENTACIÓN E INFORME TÉCNICO DE PROYECTO.

PROGRAMACIÓN III.

VISUALIZADOR INTERACTIVO DE ESTRUCTURAS DE DATOS Y ALGORITMOS.

1. INFORME TÉCNICO.

1.1 Resumen Ejecutivo.

El software analizado consiste en una aplicación de escritorio interactiva desarrollada en Python utilizando la biblioteca gráfica Tkinter. Su propósito principal es educativo, permitiendo la visualización en tiempo real y la manipulación de tres estructuras de datos fundamentales: Lista Enlazada Simple, Lista Enlazada Doble y Árbol Binario de Búsqueda (BST), complementado con una simulación del algoritmo de búsqueda en amplitud (BFS) y un módulo de autoevaluación teórica. La interfaz de usuario adopta un enfoque visual moderno mediante un diseño oscuro de estilo industrial.

1.2. Análisis de Complejidad Algorítmica (Big O).

A partir de la implementación exacta expuesta en el código fuente, se detallan las complejidades temporales y espaciales de las operaciones desarrolladas:

Estructura de Datos	Operación	Complejidad Temporal (Peor Caso)	Justificación Basada en el Código
Lista Simple	<code>insert</code>	$O(n)$	Recorre la lista secuencialmente mediante un bucle <code>while</code> hasta hallar el último nodo (<code>cur.next is None</code>). PY
Lista Simple	<code>delete</code>	$O(n)$	Requiere una búsqueda secuencial de la coincidencia del valor para reconectar los punteros. PY
Lista Doble	<code>insert</code>	$O(n)$	Al no contar con un puntero directo al final (<code>tail</code>), itera obligatoriamente desde <code>head</code> hasta el último elemento. PY
Lista Doble	<code>delete</code>	$O(n)$	Realiza una búsqueda lineal. La reconfiguración de los punteros <code>next</code> y <code>prev</code> es $O(1)$, pero localizar el nodo toma tiempo proporcional a n . PY
Árbol Binario (BST)	<code>insert</code>	$O(n)$	En un árbol balanceado es $O(\log n)$, pero el peor caso ocurre si el árbol degenera en una lista lineal. PY
Árbol Binario (BST)	<code>delete</code>	$O(n)$	Sigue la misma lógica de descenso por ramas; incluye la búsqueda del sucesor en orden si el nodo tiene dos hijos. PY
Algoritmo BFS	<code>bfs / Animación</code>	$O(n)$	Visita cada nodo exactamente una vez utilizando una cola interna para procesar el árbol nivel por nivel. PY

Nota de Optimización: Las inserciones en las listas simple y doble podrían optimizarse a $O(1)$ si la clase mantuviera una referencia directa al nodo final (tail). En la implementación actual, ambas requieren recorrer la estructura por completo.

2. DOCUMENTACIÓN DEL CÓDIGO (API REFERENCE).

2.1. Modelos de Datos (Nodos).

Node.

Representa la unidad atómica de una lista enlazada simple.

Atributos.

- ✚ **Data.** El valor almacenado por el nodo (admite enteros y cadenas).
- ✚ **Next.** Puntero o referencia al siguiente nodo del listado; inicializado en None.

DoublyNode.

Representa la unidad atómica de una lista enlazada doble.

Atributos.

- ✚ **Data.** El valor almacenado.
- ✚ **Next.** Referencia al nodo subsecuente.
- ✚ **Prev.** Referencia al nodo predecesor; facilita el recorrido bidireccional.

TreeNode.

Estructura un nodo ramificado para el árbol binario.

Atributos.

- ✚ **Data.** Valor numérico clave para el ordenamiento del árbol.
- ✚ **Left.** Puntero hacia el hijo izquierdo (valores menores).
- ✚ **Right.** Puntero hacia el hijo derecho (valores mayores o iguales).

2.2. Clases de estructuras de datos.

SingleLinkedList.

Encapsula la lógica operativa de la lista enlazada simple.

- ✚ **init()**. Inicializa el estado con un head nulo.
- ✚ **insert(data)**. Crea un nodo y lo concatena al final de la secuencia. Si la lista está vacía, lo convierte en el head.
- ✚ **delete(data)**. Localiza el primer nodo que coincida con data. Modifica el puntero next del nodo anterior para saltarse el nodo eliminado. Retorna True si la remoción fue exitosa o False en caso contrario.
- ✚ **to list()**. Método utilitario que transforma la estructura enlazada en una lista nativa de Python, simplificando la renderización gráfica.

DoublyLinkedList.

Gestiona la lista doblemente enlazada.

- ✚ **Insert (data)**. Añade un elemento al final asignando los enlaces correspondientes tanto para next como para el puntero regresivo prev.
- ✚ **Delete (data)**. Desvincula el nodo objetivo ajustando de manera segura tanto el puntero del nodo anterior como el del nodo siguiente, evitando romper la cadena.
- ✚ **to list()**. Convierte la secuencia en una lista nativa para su posterior lectura.

BinarySearchTree

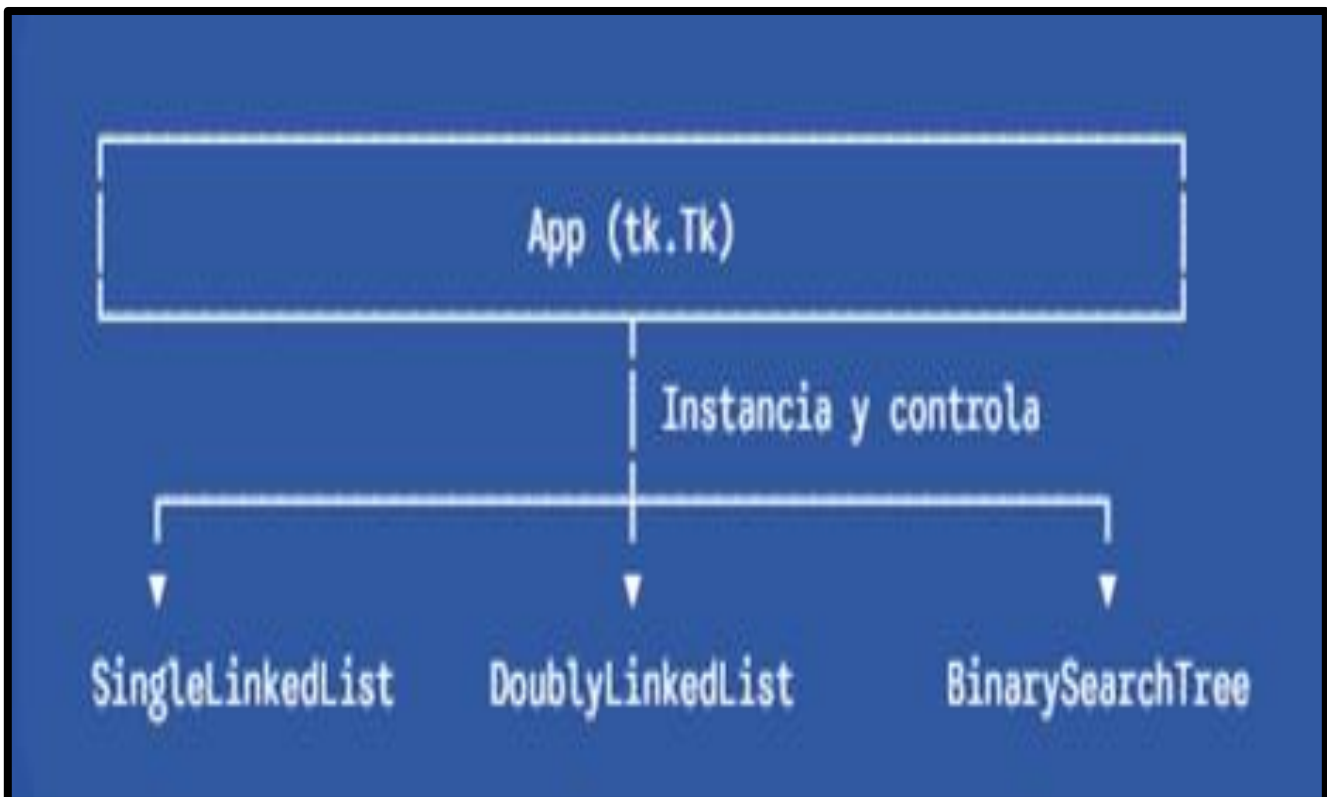
Implementa un árbol binario de búsqueda estándar.

- ✚ **insert(data) / insert (node, data)**. Algoritmo recursivo que posiciona los elementos nuevos a la izquierda si son menores que el nodo actual, o a la derecha si son mayores o iguales.
- ✚ **Delete (data) / delete (node, data)**. Elimina un nodo manejando tres escenarios posibles: nodo hoja (sin hijos), nodo con un solo hijo o nodo con dos hijos. Para este último, se localiza el sucesor en orden (el menor elemento del subárbol derecho) para su reemplazo.

✚ **bfs().** Realiza un recorrido por amplitud auxiliándose de una estructura de cola. Agrupa y retorna los elementos organizados explícitamente en sublistas por niveles jerárquicos, manteniendo los valores nulos (None) para una correcta proyección espacial en la interfaz.

2.3 Interfaz gráfica y controladores. (App).

La clase principal hereda de tk.Tk y actúa como el orquestador global de la experiencia, enlazando los eventos de la interfaz con los modelos lógicos de datos.



Paleta de color industrial: Definida estáticamente mediante constantes hexadecimales uniformes para garantizar el contraste visual (BG oscuro, componentes en PANEL y CARD, con acentos cromáticos en amarillo ACCENT para estados estándar, azul ACCENT2 para actividades asíncronas/BFS y rojo ACCENT3 para eliminaciones).

Métodos de construcción gráfica.

build_fonts(). Centraliza y parametriza las tipografías monoespaciadas (Courier New) para reforzar la estética técnica e industrial.

build ui(). Configura el contenedor raíz, las hojas de estilo de las pestañas (ttk.Style) y añade los componentes (ttk.Notebook).

make tab(). Generador genérico de paneles de interacción para las listas. Vincula las acciones de las cajas de texto y botones a los comandos lógicos correspondientes.

make tab bst(). Inicializa el entorno especializado para el árbol, declarando variables de control para la gestión de hilos de animación (_bfs_step, _bfs_idx, _bfs_job).

make tab help(). Instancia el banco interactivo de preguntas y respuestas educativas estructuradas bajo un formato dinámico de "Flashcards".

3. COMPORTAMIENTO Y OPERATIVIDAD DEL SISTEMA.

3.1. Flujo de Redibujado en Canvas.

Cada vez que el usuario altera la información (inserción o eliminación) o el tamaño de la pantalla cambia, se dispara un ciclo de refresco gráfico:

1. Se invoca canvas.delete("all") para limpiar el lienzo antiguo.
2. Se extraen los datos secuenciales actualizados mediante to_list() o se calculan las coordenadas relativas del árbol con calc_positions().
3. **Para las listas.** Se dibujan óvalos amarillos que contienen el valor del nodo, acompañados de flechas indicadoras de flujo de memoria (next en azul, prev en rojo).
4. **Para el árbol.** Se trazan primero las aristas de unión y posteriormente los nodos circulares de forma jerárquica.

3.2 Lógica de la Animación del Algoritmo BFS.

El método animate_bfs() es un motor asíncrono basado en eventos controlados:

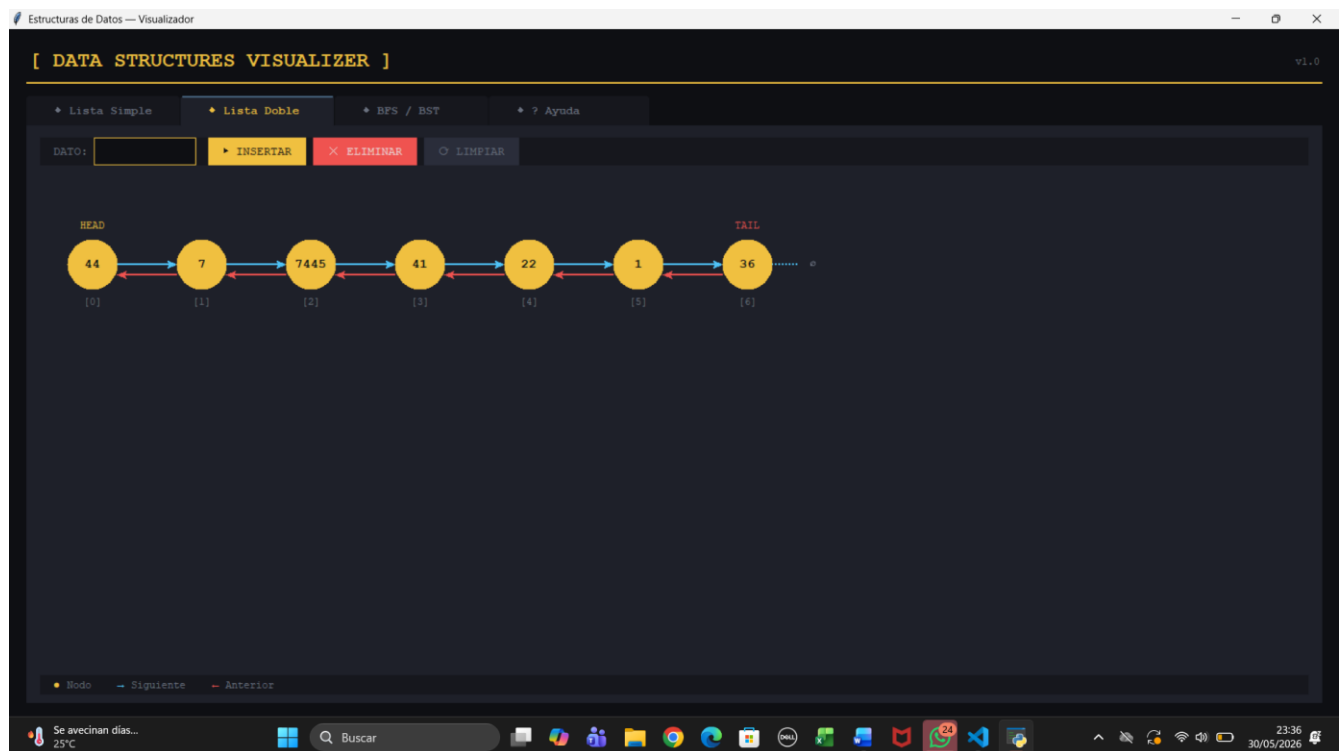
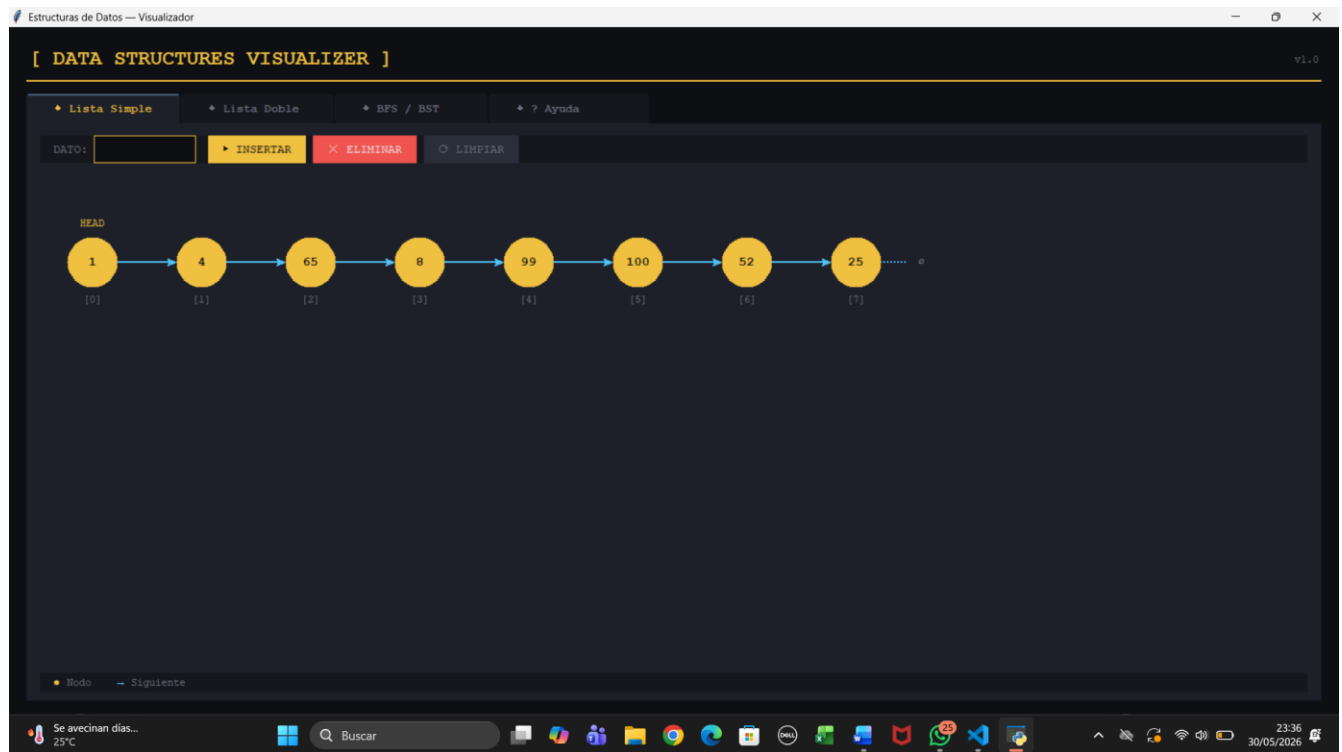
- ✚ Al pulsar el botón BFS, se obtiene el recorrido por niveles y se genera una lista aplanada de los nodos (flat).
- ✚ El controlador lee el índice actual (_bfs_idx) y redibuja el árbol pasando el valor activo a la función redraw_bst(highlight=val).
- ✚ El método muta temporalmente el color del nodo visitado a azul (ACCENT2).

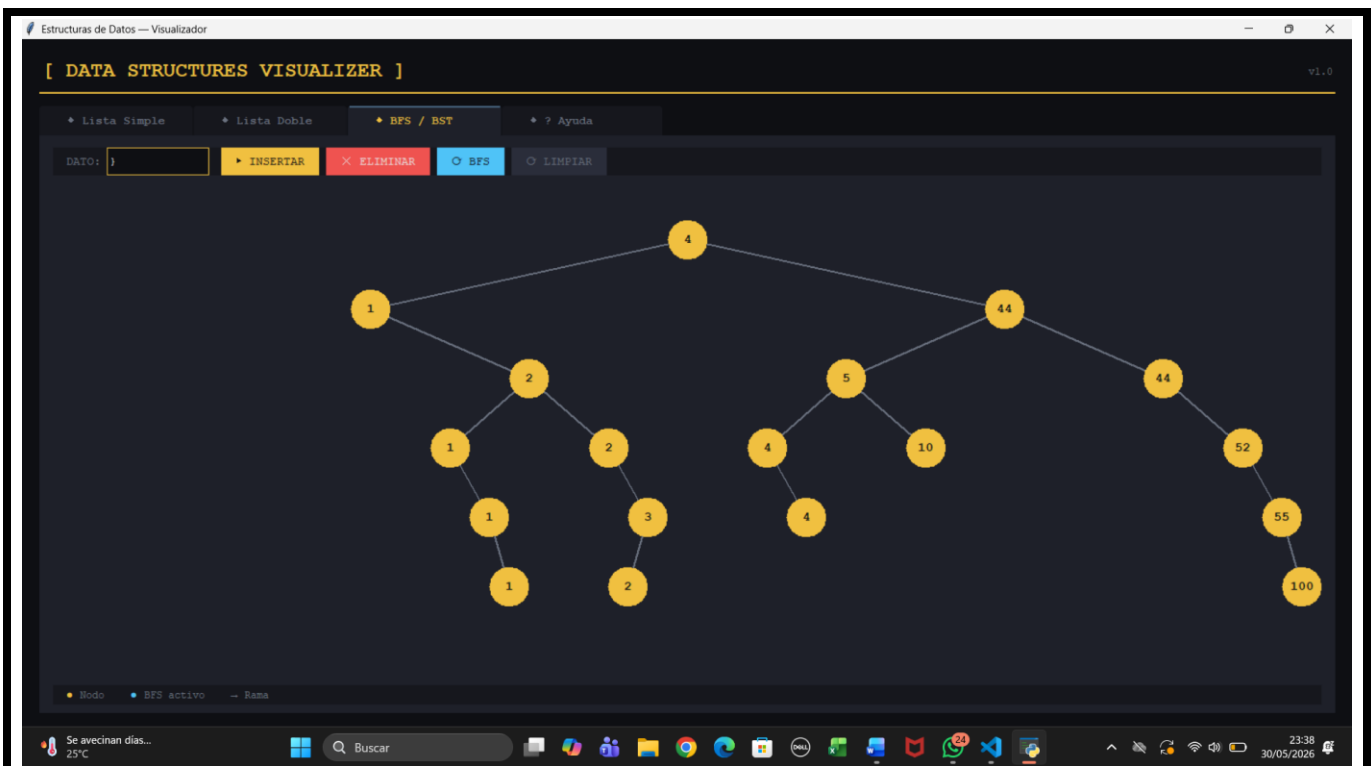
✚ Mediante `self.after(600, self._animate_bfs)`, la aplicación agenda la evaluación del siguiente nodo en los próximos 600 milisegundos, permitiendo visualizar el recorrido de manera secuencial sin congelar el hilo principal de la interfaz de usuario.

3.3 Módulo de Autoevaluación.

Diseñado para la preparación técnica avanzada, este panel segmenta cuestionarios especializados a través de un diccionario asociativo indexado por categorías. Mediante controladores de eventos (`_select_cat`, `_help_load`, `_help_reveal`), actualiza dinámicamente las etiquetas en pantalla y manipula una barra de progreso que calcula el avance proporcional del estudiante dentro de la categoría seleccionada.

CAPTURAS DE PANTALLA DE LA INTERFAZ GRÁFICA DEL CÓDIGO FUENTE.





Estructuras de Datos — Visualizador

[DATA STRUCTURES VISUALIZER] v1.0

• Lista Simple • Lista Doble • BFS / BST • **• ? Ayuda**

♦ AUTOAPRENDIZAJE — PREGUNTAS DE REPASO Practica antes de tu examen o entrevista

CATEGORÍA: **Lista Simple** Lista Doble Árbol Binario de Búsqueda (BST) BFS — Búsqueda por Amplitud Complejidades Clave

Pregunta 1 / 6

PREGUNTA

¿Qué es un nodo en una lista enlazada?

RESPUESTA

Un objeto con dos campos: 'data' (el valor almacenado) y 'next' (puntero al siguiente nodo). El último nodo tiene next = None.

◀ ANTERIOR **✓ MOSTRADA** **SIGUIENTE ▶**

Se acercan días lluviosos... ESP 22:52

